Documentation of:

qedtool

A Python package for numerical quantum information in quantum electrodynamics

Jesse Smeets[1,*], Preslav Asenov[2,†], Alessio Serafini[2,‡]

[1] *Department of Applied Physics and Science Education, Eindhoven University of Technology, P.O. Box 513, 5600 MB Eindhoven, The Netherlands*

[2] *Department of Physics and Astronomy, University College London, Gower Street, London WC1E 6BT, United Kingdom*

28 January, 2026

[*] j.smeets.physics@gmail.com
[†] preslav.asenov.20@ucl.ac.uk
[‡] a.serafini@ucl.ac.uk

Contents

1 Introduction **3**
1.1 This documentation 3

2 The `relativity` module **3**
2.1 3-vectors 3
2.2 4-vectors 4
2.3 Functions for Lorentz transformations 5

3 The `qed` module **6**
3.1 Dirac spinors 6
3.2 Real particles 6
3.3 Virtual particles 7
3.4 The `standard_scattering` function 7
3.5 Additional functions 8

4 The `qinfo` module **9**
4.1 Quantum states 9
4.2 Additional functions of `qinfo` 10

5 Standalone functions **11**
5.1 Vectors and Lorentz transformations 11
5.2 Polarizations, propagators, Dirac adjoint and the Dirac current 12

1 Introduction

`qedtool` is a Python library (licensed under the MIT license) that allows users to perform numerical relativistic quantum electrodynamics calculations. Users can construct 3-vectors, 4-vectors, photon polarizations, (anti)fermion polarizations, propagators, and compute Feynman amplitudes. By specifying initial quantum states (both pure and mixed) in polarization space, users can study the (high-energy) scattering of multi-particle quantum states. Calculable quantities include spin-correlations (Stokes parameters), the 2-particle degree of entanglement (concurrence) and cross sections. For more information regarding the package, the relevant theory, instructions, and examples, we refer the reader to our arXiv preprint.

1.1 This documentation

In this documentation, we provide an extensive list of all classes (including their methods and attributes) and functions contained within `qedtool`. For the purposes of usability, maintainability, and potential scalability, `qedtool` is organized into four main modules. Firstly, the `relativity` module contains classes and functions related to special relativity, as outlined in Section 2. Then, the `qed` module (Section 3), is used to calculate the Feynman amplitudes and various other quantities from QED. Finally, all quantities from quantum optics and quantum information are calculated with functions and classes from the `qinfo` module, described in Section 4. For the calculation of Feynman amplitudes, users can utilize standalone functions, i.e. functions that do not require the classes of `qedtool`. These are outlined in Section 5.

2 The `relativity` module

The module named `relativity` describes general 3-vectors, 4-vectors, operations thereon and relevant functions. 3- and 4-vectors have their own classes, namely `ThreeVector` and `FourVector`. The `ThreeVector` class can be used to describe e.g. boost and angle vectors for Lorentz transformations. The `FourVector` class can be used to describe e.g. 4-momenta and 4-polarizations, which can be boosted to other inertial reference frames.

2.1 3-vectors

Objects that are instances of the class `ThreeVector` represent 3-vectors. The constructor parameters for the creation of `ThreeVector` instances are

- `c1`, `c2`, `c3`: `float` or `complex`
  The (spatial) components of the 3-vector. Their meaning depends on the `coordinates` argument.

- `coordinates`: `str`
  The coordinate system that specifies the meaning of `c1`, `c2` and `c3`. If `coordinates` is `"Cartesian"`, then `c1` through `c3` denote (x, y, z). If it equals `"cylindrical"` or `"spherical"`, they denote (ρ, ϕ, z) or (r, θ, ϕ) as defined in the main article.

Users may add, subtract and multiply instances of `ThreeVector`, where multiplication will return the Euclidean inner product. Moreover, scaling of instances is possible, i.e. multiplication and division of an instance of `ThreeVector` with a scalar. `ThreeVector` contains various attributes;

- `vector`: `ndarray` of shape `(3,)`
  The 3-vector expressed as an `ndarray` of shape `(3,)`, in Cartesian components.

- `cartesians`: `tuple` of shape `(3,)`
  The Cartesian x-, y- and z-components of the 3-vector.

- `cylindricals`: `tuple` of shape `(3,)`
  The cylindrical ρ-, ϕ- and z-components of the 3-vector. If the vector contains complex components, then `cylindricals` equals `None`.

- `sphericals`: `tuple` of shape `(3,)`
  The spherical r-, θ- and ϕ-components of the 3-vector. If the vector contains complex components, then `sphericals` equals `None`.

The `ThreeVector` class also contains two methods:

- `dot(mat, vec)`
  Calculates the product of a 3×3 matrix `mat`, an `ndarray` of shape `(3, 3)`, with the `ThreeVector` instance `vec`. It returns an instance of `ThreeVector`.

- `beta(pmu)`
  Takes the 4-momentum `pmu`, i.e. an instance of `FourVector` (see Section 2.2) and returns the 3-velocity $\boldsymbol{\beta}$, an instance of `ThreeVector`.

While `dot` is purely meant for matrix multiplication, the Euclidean inner product has its own function, which is utilized in the multiplication of two `ThreeVector` instances (using the standard Python syntax; an asterisk).

2.2 4-vectors

The `FourVector` class allows the user to define 4-vectors. The constructor parameters are

- `c0`, `c1`, `c2`, `c3`: `float` or `complex`
  The four components that specify the 4-vector. `c0` always signifies the time-component, whereas `c1` to `c3` are the spatial components. The meaning of the latter three is specified by the `coordinates` argument.

- `coordinates`: `str`
  The coordinate system that specify the meaning of `c1` through `c3`, see the `ThreeVector` constructor parameters in Section 2.1.

Similar to 3-vectors, 4-vectors can be added to or subtracted from each other, as well as scaled by a scalar. Multiplying two 4-vectors returns the Lorentzian inner product. The `FourVector` class contains the following attributes:

- `vector`: `ndarray` of shape `(4,)`
  The 4-vector expressed as an `ndarray` of shape `(4,)`, in Cartesian components.

- `cartesians`: `tuple` of shape `(4,)`
  The Cartesian t-, x-, y- and z-components of the 4-vector.

- `cylindricals`: `tuple` of shape `(4,)`
  The cylindrical t-, ρ-, ϕ- and z-components of the 4-vector. If the vector contains complex components, then `cylindricals` equals `None`.

- `sphericals`: `tuple` of shape `(4,)`
  The spherical t-, r-, θ- and ϕ-components of the 4-vector. If the vector contains complex components, then `sphericals` equals `None`.

One of the advantages of using the `FourVector` class, is that if the spherical components are defined as the input, then for $|\mathbf{p}| = 0$, the spherical angles are memorized. This is especially useful when defining polarizations at zero momentum. `FourVector` also contains a set of methods that allows the user to construct 4-vectors that are customary within QED,

- `dot(mat, vmu)`
  Calculates the matrix product of `mat`, an `ndarray` of shape `(4, 4)`, and the `FourVector` instance `vmu`. It returns an instance of `FourVector`.

- `polarization(handedness, pmu, conjugate=False)`
  Takes the following parameters as inputs: the 4-momentum p of a photon (`pmu`, an instance of `FourMomentum`), the photon's `handedness` $\lambda \in \{-1, +1\}$ (of type `int`), and the Boolean parameter `conjugate`. If `conjugate` is `False` (which is the default value), then `polarization` returns the 4-polarization $\epsilon_\lambda(p)$ of the considered photon, which is an instance of the `FourVector` class. If `conjugate` is `True`, the complex conjugate of $\epsilon_\lambda(p)$.

- `dirac_current(psi_1, psi_2)`
  Takes two Dirac spinors `psi_1` and `psi_2` as inputs, instances of the `DiracSpinor` class (see Section 3.1) or the `RealParticle` class (see Section 3.2). It returns a Dirac current $j^\mu = \bar{\psi}_1 \gamma^\mu \psi_2$, which is an instance of `FourVector`. Here, `psi_1` should already have its Dirac adjoint taken (i.e. `psi_1.adjoint = True`) and `psi_2` not.

2.3 Functions for Lorentz transformations

The `relativity` module also includes functions that are relevant for Lorentz transformations:

- `lorentz_factor(pmu)`
  Calculates the Lorentz factor $\gamma_{\boldsymbol{\beta}}$ (`float`) of the 4-momentum `pmu` (an instance of the `FourVector` class that is timelike).

- `rapidity(pmu)`
  Calculates the rapidity η by first obtaining $\gamma_{\boldsymbol{\beta}}$ using `lorentz_factor`. Then
  $$\eta = \operatorname{arccosh} \gamma_{\boldsymbol{\beta}}$$
  equals the rapidity, returned as a `float`. `pmu` is a timelike `FourVector`.

- `pseudorapidity(pmu, n=None)`
  For a 4-vector p and specified direction $\mathbf{n}$ (`pmu` and `n`), it calculates the pseudorapidity
  $$y_{\mathbf{n}}(p) = -\ln\left(\tan\tfrac{\theta}{2}\right)$$
  where θ is the angle between $\mathbf{p}$ and $\mathbf{n}$. Here, `n` can either be an `ndarray` of shape `(3,)` or a `ThreeVector`. By default, `n` is taken to be in the z-direction. `pmu` is a timelike `FourVector`. The return is a `float`.

- `boost(obj, beta)`
  Actively boosts an object `obj` by a boost vector `beta` (`ThreeVector`). Here, `obj` can be an instance of the following classes: `QuantumState`, `FourVector`, `DiracSpinor`, `RealParticle` or `VirtualParticle`. Each type of object is boosted in the corresponding representation. For real particles, this entails their polarization and momentum. For virtual particles, their propagators and momenta are boosted. The returned object is a deep copy of `obj` with boosted properties.

- `rotation(obj, angle_vec)`
  Similar to `boost`. Here `angle_vec` ($\boldsymbol{\theta}$) is a `ThreeVector`. Instead of boosting objects, it actively rotates them along the axis $\boldsymbol{\theta}/|\boldsymbol{\theta}|$ by an angle $|\boldsymbol{\theta}|$.

3 The `qed` module

This module carries classes and functions that are primarily for computing Feynman amplitudes. The three classes contained within `qed` are `DiracSpinor`, `RealParticle` and `VirtualParticle`, outlined in Sections 3.1 through 3.3. Moreover, the `qed` module has the `standard_scattering` function, which allows users to calculate the introduced quantum information quantities for standard 2-to-2 particle QED scattering processes, for intervals of $|\mathbf{p}|$, θ and ϕ in the CM frame using a single command. This function is discussed in Section 3.4.

3.1 Dirac spinors

The `DiracSpinor` class constructs momentum space Dirac spinors for external fermions. To construct a Dirac spinor, users need to specify

- `handedness: int`
  The handedness of the spinor, must be ± 1.
- `pmu: FourVector`
  The on-shell 4-momentum of the (anti)fermion.
- `anti: bool`
  Whether the Dirac spinor corresponds to a fermion or antifermion. The default value for `anti` is `False`.
- `adjoint: bool`
  Whether the Dirac adjoint should be taken or not. The default value for `anti` is `False`.

After specifying the parameters above, the constructor returns a `DiracSpinor`. Attributes of `DiracSpinor` instances are `adjoint`, `handedness` and `bispinor`. `bispinor` is the `ndarray` that represents the actual Dirac spinor. The `DiracSpinor` class contains one method, namely

- `dirac_adjoint(psi)`
  Takes the Dirac adjoint of Dirac spinor `psi`, an instant of `DiracSpinor`. The return is an adjointed Dirac spinor $\bar{\psi} = \psi^\dagger \gamma^0$ of type `DiracSpinor`.

3.2 Real particles

The `RealParticle` class allows users to define measurable particles that are in momentum-helicity eigenstates. Directly creating a particle with the `RealParticle` constructor allows users to define custom external particles. In this fashion, properties such as the mass, charge, and polarizations must be assigned by the user. However, when real particles are created through the constructor, most of its attributes (except for the input arguments) are `None`. Therefore, users are encouraged to define particles with the methods of `RealParticle`, for example `RealParticle.electron`. Other methods are `positron`, `photon`, `muon`, and `antimuon`. The input parameters for these methods are

- `handedness: int`
  The particle's handedness.
- `pmu: FourVector`
  The on-shell 4-momentum of the particle.
- `direction: str`
  Whether the particle enters or exits the scattering event, i.e. `"in"` or `"out"` respectively.

The attributes of this class are

- `species: str`
  The type of particle.
- `four_momentum, direction`
  Equivalent to the constructor parameters `pmu` and `direction`.
- `spin: float`
  The intrinsic spin quantum number s of the particle.
- `mass: int`
  The particle's rest mass. Units are MeV by default, and changed with the `constant` function in Section 3.5 for other mass units.
- `charge: float`
  The charge expressed in elementary charge units. For an electron, `charge` is -1.
- `polarization: DiracSpinor` or `FourVector`
  The polarization of the particle. For photons it is the 4-polarization $\epsilon_\lambda(p)$ (or its complex conjugate, depending on `direction`). For fermions it is the momentum-helicity Dirac spinors $u_\sigma(p)$ and $v_\sigma(p)$ (or their Dirac adjoints, depending on `direction`).

3.3 Virtual particles

Instances of `VirtualParticle` can be defined by the `species` and `pmu` parameters (these have the same meaning as for the `RealParticle` class). However, users are encouraged to utilize the methods within `VirtualParticle`; `electron`, `positron`, etc. Virtual particles have attributes

- `species, four_momentum`
  Equivalent to the constructor parameters.
- `virtuality: float`
  If q is the 4-momentum of the virtual state, then $Q^2 = -q^2$ is its virtuality. Since virtual states are mostly off-shell, $q^2 \neq m^2$.
- `propagator: float`
  The momentum space Green's function of the virtual state.
- `mass: int`
  The particle's rest mass. Units are MeV by default, and changed with the `constant` function in Section 3.5 for other mass units.

3.4 The `standard_scattering` function

The `standard_scattering` function is a compact function in the `qed` module, which calculates the main quantum-informational quantities offered by `qedtool` for six common 2-to-2 particle QED scattering processes in the CM frame and at tree level. These processes are Compton, Bhabha, Møller, and electron-muon scattering, as well as electron-positron annihilation and the electron-positron to muon-antimuon annihilation-creation process. The inputs of `standard_scattering` are

```
qtl.standard_scattering(in_state, scattering, momentum,
        theta, phi=None, filename=None, projection=None,
        dp=False, dcs=False, c=False, stokes=False,
        deg_pol=False, amplitudes=False, out_state=False)
```

The `in_state` input of `standard_scattering`is an instance of the `QuantumState` class. It is defined as the incoming quantum state in the CM frame. The `scattering` string input specifies the selected scattering process and can be selected as one of the following:

- `"compton"`
- `"bhabha"`
- `"moller"`
- `"electron_muon"`
- `"electron_positron_annihilation"`
- `"electron_positron_to_muon_antimuon"`

A limitation of `standard_scattering` is that it considers scattering only in the scenario where the 4-momentum of the incoming electron (all 6 considered processes include at least one incoming electron) has a 3-momentum parallel to the positive z-axis. The other scattering particle must have an incoming 3-momentum parallel to the negative z-axis. Output quantities are then calculated over the range of specified `momentum`, `theta`, and `phi` values, input as arrays. If one of these inputs is selected as `None` instead, then the corresponding physical parameter is set to zero. Users have the option to automatically save the function's output in the form of a `pickle` file, entitled with the optional `filename` string, which is `None` by default. If `filename` is `None`, then no file is saved.

`standard_scattering` returns a dictionary of any of the quantities whose Boolean input is selected as `True`. This dictionary is of the form `dict["key"][momentum][theta][phi]`. A `projection` quantum state can be specified as a particular instance of the `QuantumState` class. Then the true output state resulting from the particle evolution is projected onto the `projection` and all output variables are adjusted accordingly. This input is `None` by default. If `dp=True`, an array of the differential probability values $\partial_\Pi \mathcal{P}$, is included in the dictionary. To obtain the differential cross section, the `dcs` keywords needs to equal `True`. If `c=True`, the concurrence values for all input momentum, polar angle, and azimuthal angle values are included in the dictionary. Similarly, all two-particle Stokes parameters $S_{\mu\nu}$ and the two-particle degree of polarization are included if `stokes` and `deg_pol` are `True`. If `amplitudes=True` (which is its default value), then arrays of polarized scattering amplitude values are included in the dictionary. If `out_state=True`, then an array of outgoing states (one instance of the `QuantumState` class for each input $|\mathbf{p}|$, θ, and ϕ values) is included in the output dictionary. The dictionary key for each value is the same as the corresponding input variable name, except the Stokes parameter arrays, which have keys `"s01"` for S_{01}, `"s02"` for S_{02}, etc., and the Feynman amplitudes' arrays, which have keys `"ll_to_ll"` for $\mathcal{M}_{\mathrm{LL}\to\mathrm{LL}}$, `"ll_to_lr"` for $\mathcal{M}_{\mathrm{LL}\to\mathrm{LR}}$, etc.

3.5 Additional functions

The `qed` module consists of a set of additional important functions. Users can specify quantities and units using

> `constant(quantity, units=None)`
> Returns a quantity as a `float` in the specified `units` (`str`). The keyword `quantity` can be one of the following: `"electron mass"`, `"muon mass"`, `"elementary charge"`, or `"fine structure constant"`. These can be expressed in the following units: `"meV"`, `"eV"`, `"keV"`, `"MeV"`, or `"GeV"`. If `units` equals `None`, then MeV are taken. Moreover, after users specified e.g. the electron mass, then the same units are automatically employed in the `RealParticle` and `VirtualParticle` classes (see Sections 3.2 and 3.3).

Contractions of 4-vectors with the gamma matrices are computed with

`slashed(vmu)`
For the 4-vector `vmu`, which can be an `ndarray` of shape `(4,)` or a `FourVector`, it returns the contraction with the gamma matrices $\not{v}$ as an `ndarray` of shape `(4, 4)`.

To calculate Feynman amplitudes, it is customary to utilize the `handedness_config` function, which generates configurations of handednesses:

`handedness_config(n, fixed=None, fixedval=None)`
Returns all handedness configurations. Here `n` (`int`) denotes the number of particles. The number of handedness configurations is 2^n. Users can fix the handedness of particles using the `fixed` and `fixedval` arguments, which must be of type `int` (if only one handedness needs to be fixed) or `array_like`. In `fixed`, the indices ($\leq n-1$) of the fixed handedness values are specified. `fixedval` contains the handedness values themselves that are to be fixed (± 1). By default, no handedness values are fixed.

Finally, users can quickly create empty arrays, print `for`-loop progress and save data using the following functions:

- `empty_lists(n)`
  Constructs a Python list that contains `n` empty lists.

- `progress(idx, length)`
  Prints the progress of a `for`-loop. Inside a `for`-loop, where `idx` (of type `int`) goes over the elements of an array of size `length` (which is an `int`), the `progress` function prints the percentage of completed `for`-loop, rounded to three decimals.

- `save_data(filename, keys, data)`
  Constructs a dictionary and saves it as a `pickle` file. Here `filename` is of type `str`, `keys` is a 1-dimensional `array_like` input that contains the keys for the dictionary. `data` must also be `array_like`; for the `j`-th key, `data[j]` will be stored under that key.

4 The `qinfo` module

The `qinfo` module contains all classes and functions for the quantum informational and quantum optics calculations. This includes constructing multi-particle quantum states, retrieving Stokes parameters and the concurrence. The only class in this module is the `QuantumState` class.

4.1 Quantum states

When defining quantum states, e.g. for the initial scattering state, users can create instances of the class `QuantumState`. Instances of this class describe the polarization quantum state. Quantum states can be added and subtracted from each other, and they scaled with complex numbers with the standard Python syntax to make superpositions. The `QuantumState` class disposes over four attributes,

- `bra`, `ket` : `ndarray`
  The bra and ket representation of the quantum state in the $\{\mathrm{L}, \mathrm{R}\}^{\otimes n}$ basis for an n-particle state. If the state is mixed, then these attributes are `None`.

- `rho` : `ndarray`
  The density matrix of the quantum state. Unlike `bra` and `ket`, `rho` is always defined.

- `four_momentum: FourVector`
  In the current release, these 4-momenta only play a role in Lorentz transformations of quantum states. If the `QuantumState` instance is a single-particle momentum eigenstate, then `pmu` is its 4-momentum. Otherwise it equals `None`.

The main ingredients for creating (pure) polarization quantum states are single-particle states, the tensor product to form multi-particle states and the concept of superposition. These methods, together with methods for constructing mixed states are all contained within the `QuantumState` class. Below is a list of all methods:

- `single(pmu, polarization)`
  Returns a single-particle momentum-helicity eigenstate, an instance of `QuantumState`. Here, `polarization` is of type `str`, which can be either `"L"`, `"R"`, `"H"`, `"V"`, `"D"` or `"A"` (meaning "left", "right", "horizontal", "vertical", "diagonal" and "antidiagonal" respectively). The 4-momentum `pmu` is of type `FourVector`, which plays a crucial role in the Lorentz transformation of single-particle momentum-helicity eigenstates. `pmu` can be set to `None`. Then, the quantum state cannot be Lorentz transformed. This is especially useful when users are in fixed inertial reference frames, like in the `standard_scattering` function (see Section 3.4).

- `mixed(states, w)`
  Returns an instance of `QuantumState`, which can be either a pure or a mixed state. Here, `w` is an ndarray of size (n,) defining the n classical probabilities of occupying the n quantum states in the ndarray `states`. If `states` is the set $\{|\psi_j\rangle\}$ and `w` is the set of classical probabilities $\{w_j\}$, then `mixed` returns the quantum state represented by density operator $\sum_j w_j|\psi_j\rangle\langle\psi_j|$. The attributes `bra` and `ket` of the output are `None`.

- `out_state(in_state, amplitudes)`
  Returns the momentum-projected out-state with the density operator, corresponding to an initial state `in_state` and polarized scattering amplitudes called `amplitudes`. Here in_state is an instance of the QuantumState class and amplitudes is an array_like object of size (4,4). The output is an instance of `QuantumState`. The trace of the corresponding density matrix is normalized such that $\mathrm{tr}\,\rho = \partial_\Pi \mathcal{P}$. The parameter `in_state` is also an instance of `QuantumState`. The first index in the `amplitudes` array denotes the final-state handedness and the second index is the initial-state handedness. For 2-to-2 scattering, helicity eigenvalues are indexed according to the following order: $\{\mathrm{L},\mathrm{R}\}^{\otimes 2} = \{\mathrm{LL},\mathrm{LR},\mathrm{RL},\mathrm{RR}\}$.

4.2 Additional functions of `qinfo`

The `QuantumState` class from Section 4.1 has methods that return new quantum states. There is an additional set of functions, that allows the user to determine quantum optics and entanglement quantities from quantum states. We outline these functions below:

- `differential_probability(out_state, projection=None)`
  Calculates the (projected) differential scattering probability $\partial_\Pi \mathcal{P}_{(\varrho)}$ from the post-scattering state. As mentioned in the main article, it is possible to project the final scattering state onto some other state `projection`. Here, `out_state` and `projection` are both instances of the `QuantumState` class. If `out_state` is an n-particle state, then so must `projection` be.

- `diff_cross_section(pmu_1, pmu_2, out_state, projection=None)`
  Calculates the (projected) 2-to-2 particle differential cross section from the post-scattering

state and the initial particles' 4-momenta `pmu_1` and `pmu_2` (instances of `FourVector`). Here, `out_state` and `projection` are both instances of the `QuantumState` class. If `out_state` is an n-particle state, then so must be `projection`.

- `concurrence(state)`
  Obtains the concurrence of a two-particle state. The input `state` is an instance of `QuantumState` that represents a state that is not necessarily trace normalized. Namely, `concurrence` will normalize the state to $\mathrm{tr}\,\rho = 1$ and calculate the concurrence. The eigenvalues of $Q \equiv \rho\tilde{\rho}$ with the spin-flipped density matrix $\tilde{\rho} \equiv (\sigma_2 \otimes \sigma_2)\rho^*(\sigma_2 \otimes \sigma_2)$, which are required for the concurrence, are determined with `numpy.linalg.eig`.

- `stokes_parameter(state, l)`
  Calculates the Stokes parameters corresponding to `state`, an n-particle instance of the `QuantumState` class that is not necessarily normalized. Here `l` is `array_like` that represent the Stokes tensor indices $\{\mu_1, ..., \mu_n\}$. Before calculating the Stokes parameters, `stokes_parameter` normalizes the density matrix of `state` to $\mathrm{tr}\,\rho = 1$.

- `degree_polarization(state)`
  Returns the single- or two-particle degree of polarization. The input `state` is an instance of the `QuantumState` class. The density matrix ρ of `state` is automatically normalized to $\mathrm{tr}\,\rho = 1$.

- `inner_product(state_1, state_2)`
  Calculates the overlap of `state_1` with `state_2` in polarization space, both instances of the `QuantumState` class with equal particle numbers. It returns the dot product of `state_1.bra` with `state_2.ket`, i.e. $\langle\psi_1|\psi_2\rangle$, which is of type `complex`. Here $\langle\psi_1|$ and $|\psi_2\rangle$ denotes `state_1.bra` and `state_2.ket`. It does not take into account the orthogonality in momentum space, nor the species of particles the states represent.

- `density_matrix(state)`
  Constructs the density matrix, a square `ndarray`, for a specified state. Here `state` is an instance of `QuantumState`.

5 Standalone functions

Here we highlight the functions that do not require the use of the `qedtool` classes. They are especially useful in performing operations on 3-vectors and 4-vectors, and for computing Feynman amplitudes.

5.1 Vectors and Lorentz transformations

There are two standalone functions for Lorentz transformations of 4-vectors and fields; they return the 4×4 Lorentz transformation matrices in the $(\frac{1}{2}, \frac{1}{2})$ and $(\frac{1}{2}, 0)\oplus(0, \frac{1}{2})$ representations:

- `boost_matrix(beta, representation)`
  Returns the active Lorentz boost matrix associated with the input `beta`, an `ndarray` of shape `(3,)` or an instance of `ThreeVector`, and the input `representation`, which is of type `str`. If `representation = "4-vector"`, then `boost_matrix` returns the transformation matrix required to boost a four-vector by `beta` as an `ndarray` of shape `(4,4)`. If `representation = "Dirac spinor"`, then `boost_matrix` returns the `ndarray` of shape `(4,4)` required to boost a Dirac spinor by `beta`.

- `rotation_matrix(angle_vec, representation)`
  Similar to `boost_matrix`. Instead of boost matrices, it returns active rotation matrices associated with different mathematical objects. `representation` is an iput of type `str`, which can be equal to `"three-vector"`, `"four-vector"`, or `"Dirac spinor"`. `angle_vec` is an input of type `ndarray` or is an instance of the `ThreeVector` class. It gives the three angles defining the rotation matrix.

Lorentzian and Euclidean products of instances of type `ndarray` can be computed with

- `lorentzian_product(vmu_1, vmu_2)`
  Returns the Lorentzian product of `vmu_1` and `vmu_2`, which can both be of type `ndarray` and shape `(4,)` or be instances of `FourVector`.

- `euclidean_product(vec_1, vec_2)`
  Returns the Euclidean product of `vec_1` and `vec_2`, which are both of type `ndarray` and shape `(3,)`.

The standalone equivalents of the `cartesians`, `sphericals` and `cylindricals` attributes of the `ThreeVector` and `FourVector` classes are the functions

- `spherical_components(vector)`
  Returns the spherical components of `vector`, which is an `ndarray` of shape `(3,)` for 3-vectors or `(4,)` for 4-vectors.

- `cylindrical_components(vector)`
  Returns the cylindrical components of `vector`, which is an `ndarray` of shape `(3,)` for 3-vectors or `(4,)` for 4-vectors.

- `cartesian_components(vector)`
  Returns the Cartesian components of `vector`, which is an `ndarray` of shape `(3,)` for 3-vectors or `(4,)` for 4-vectors.

for `vector` of type `ndarray`.

5.2 Polarizations, propagators, Dirac adjoint and the Dirac current

The standalone functions in `qed` are

- `fermion_polarization(handedness, pmu)`
  Returns the momentum-helicity Dirac eigenspinor of a fermion as an `ndarray` that has shape `(4,)`, given `handedness`, which is an `int` equal to `1` or `-1`, and `pmu` is an `ndarray` of shape `(4,)`.

- `antifermion_polarization(handedness, pmu)`
  See the `fermion_polarization` function.

- `photon_polarization(handedness, pmu)`
  Constructs the 4-polarization of a photon from its handedness and 4-momentum `pmu`, which is an `ndarray` of shape `(4,)`. The function returns an `ndarray` of shape `(4,)`.

- `fermion_propagator(pmu, m)`
  Returns the momentum-space Green's function $\tilde{G}(p)$ of a massive fermion with mass `m` (an input of type `float`) as an `ndarray` of shape `(4, 4)`.

- `photon_propagator(pmu)`
  Returns the photon momentum space Green's function $\tilde{D}_{\mu\nu}(p)$ as an `ndarray` of shape `(4, 4)`.

- `dirac_current(psi_1, psi_2)`
  Constructs the Dirac current $j^\mu = \psi_1 \gamma^\mu \psi_2$ for Dirac spinors `psi_1` and `psi_2`. Here `psi_1` and `psi_2` can be of type `ndarray` with shape `(4,)` or instances of `DiracSpinor`. The function returns an `ndarray` of shape `(4,)`. To obtain a Dirac current as a `FourVector`, see the `FourVector.dirac_current` method in Section 2.2.

- `dirac_adjoint(psi)`
  Takes the Dirac adjoint of a Dirac spinor `psi`, which can be an `ndarray` of shape `(4,)` or a `DiracSpinor`. If `psi` is an instance of `DiracSpinor`, then the adjoint can only be taken if `psi.adjoint` is `False`. The output is a `ndarray` of shape `(4,)`.